The previous chapter ended with a question. Levenberg-Marquardt converges to a stationary point of the cost funciton, and which stationary point it reaches is determined by where it begins. The solver is only as good as its initial guess. This chapter builds a Neural-Network based initialiser which then provides an inital guess for LM to allow it to converge to the correct stationary point.

Section 0.1 implements the forward map in Python and fixes the sampling scheme. Section **??** chatacterises the solver: its accuracy when initialised near the truth, its failure rate when initialised at a fixed point, and the basins of attraction. Section **??** establishes the importance of choosing a good method to quantify "closeness", particularly with respect to angle. Section **??** determines the limits of a Neural-Network based solver and motivates the need for LM refinement. Section **??** joins the two halves and compares three solvers on a common test set: the network alone, a LM solver, and the network used to initailse the solver.

Section

0.1 Data genration

0.1.1 Anser Simulation in Python

We simulate the Anser system in python, developing on a MATLAB simulation by Jaeger et al. [**?**]. [1] The pipeline proceeds as follows:

Coil geometry First we generate the vertex coordinates for a square spiral PCB emitter coil. This is done in a local coordinate system and then copies are rotated and translated to place them in the global coordinate system.

Each coil is a square which spirals in, it has N turns per layer, the side-length of the outermost square is l, the trace width is w, there is a line to line spacing of s and the thickness of the PCB is z_{thick}. After the coil has done N_{turns} turns on the upper layer, it is connected with a via to the lower layer where the coil spirals out again.

Conversion from Local to Global Coordiantes As all of the coils are the same shape, only one coil is generated, then it is copied a number of times into different places in space. The module selects an axis about which the rotation takes place. This is represented as an integer $(0, 1, 2) - (x, y, z)$.

[1] The implementation is available at `https://github.com/pmccarthy3901/ansermodelling`. A mapping from the sections of this chapter to the corresponding modules is given in Appendix **??**.

Variable	Min (mm)	Max (mm)
x	-125	125
y	-125	125
z	1.6	250

Table 1: Bounds of the padded sampling volume.

This works nicely because the rotation matrix can be generated using an even permutation of the axes. The module then applies the permuted rotation matrix to a copy of the input vector and then applies a translation.

Field Calculation The calculation of the magnetic field generated by the coils is done in `field_coil_calc`. It uses the linearity of the biot savart line integral to calculate the total magnetic field due to the coils at an observer point as the sum of the contributions of each straight-line segment. It then implements equation **??** to calculate the magnetic field.

Objective Function The objective function takes in the real observed flux, and returns the difference between it and the flux calculated using the simulation. It calculates the simulated flux as:

$$\Phi = H_{sim} \cdot \hat{n}_{obs} \tag{1}$$

where:

- H_{sim} is the simulated magnetic field
- $\hat{n}_{obs}$ is the unit vector normal to the observer.

0.1.2 Sampling

When generating data, it is important that our sampling space is uniform. For position, this is easy, we sample uniformly in the bounds of the box. The angle, however requires a bit more consideration.

0.1.2.1 Sampling the position

The anser system operates in a 25x25x25cm box.

Since the sensor coil is on a PCB that is 1.6mm thick, the z value has a minimum of $z = 0.0016$m. Therefore we sample uniformly within the bounds shown in Table 2

Variable	Min	Max
$\cos\theta$	-1	1
ϕ	0 rad	2π rad

Table 2: Bounds of the oritation sampling volume.

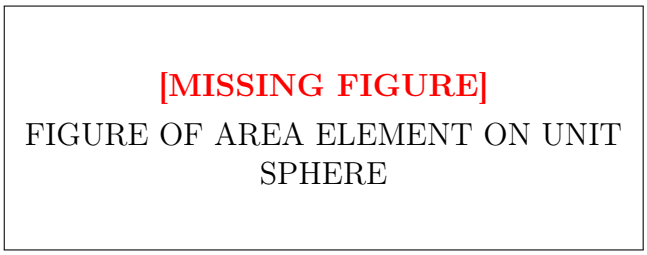

Figure 1: FIGURE OF AREA ELEMENT ON UNIT SPHERE

0.1.2.2 Sampling the orientation

Consider an area element dA on the unit sphere, seen in figure **??**. Then the area element is $dA = \sin\theta d\theta d\phi$. Therefore, to uniformly sample on the sphere we must not sample uniformly in θ, but rather in $\cos\theta$. The orientation sampling is shown in Table **??**.::

0.1.3 Dataset size, split, and normalisation

The dataset consists of $N = 100000$ pose-flux pairs. The fluxes are the model input and the pose is the regression target. The data is split 80/20 into training and test sets, and the test set is never seen during training. Because the flux magnitudes vary over several orders of magnitude across the volume, the inputs are normalised using the mean and standard deviation of the training set. The same statistics are applied to the test set to prevent data leakage.

0.1.4 Noise model

A real sensor measurement will always have some amount of noise. We model this as additive Gaussian noise on the flux vector,

$$\tilde{\mathbf{\Phi}} = \mathbf{\Phi} + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2 I_8), \tag{2}$$

and quantify the noise level by the signal-to-noise ratio

$$\mathrm{SNR} = \frac{\Phi_{\mathrm{rms}}}{\sigma}, \qquad \Phi_{\mathrm{rms}} = \frac{1}{\sqrt{8}} \left\|\mathbf{\Phi}\right\|_2 . \tag{3}$$

0.2 Representing the pose

The pose contains two angles, and how they are represented as regression targets matters more than it first appears. Regressing against θ and ϕ directly has two problems. First, ϕ is periodic: $\phi = 0.01$ and $\phi = 2\pi - 0.01$ are only seperated by 0.02 rad, however a naive loss would heavily punish such a situation. The second problem is with theta: at the poles, theta is not defined.

For these reasons, it becomes natural to represent the orientation not in terms of angles, but in terms of the unit normal itself:

$$\hat{n}(\theta, \phi) = \begin{bmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{bmatrix} \tag{4}$$

The normal vector is smooth, unique, and has no wrap-around. We therefore train networks whose target is the vector:

$$t = (x, y, z, n_1, n_2, n_3) \tag{5}$$

where n_i is the unit normal in the ith direction.

0.3 Error metrics

In order to quantify how "good" a particular model is, we need to define our errors. Since the pose has two components: position and orientation, we define an error on each.

0.3.0.1 Positional Error

The positional error e_p is defined using the euclidean distance between the true position, x_{true}, and the predicted position, x_{pred}.

$$e_p := \|x_{true} - x_{pred}\|_2 \tag{6}$$

0.3.0.2 Angular Error

The angular error, $e_{\hat{n}}$, is defined in relation to the unit normal orientations, where $\hat{n}_{true}$ is the true unit normal, and $\hat{n}_{pred}$ is the predicted unit normal:

$$e_{\hat{n}} := \arccos\left(\hat{n}_{true} \cdot \hat{n}_{pred}\right) \tag{7}$$

0.4 The Levenberg-Marquardt solver

The previous chapter derived the Levenberg-Marquardt update function **??**. This section describes the implementation and parameter choice.

0.4.1 Implementation

The solver is implemented in NumPy (`models/LM_solve.py`). Its components are as follows:

Jacobian. As discussed in the previous chapter, we approximate the jacobian using finite difference with $h = 10^-5$ **??**

Scaling. Following Marquardt's implementation [**?**], the damping matrix is given by:

$$D = \text{diag}(\max(\text{diag}(J^\top J)), 10^{-12}) \tag{8}$$

where the floor of 10^{-12} guards against a zero diagonal entry making D singular, and diag is the function that operates on matrices by setting all off-diagonal elements to zero and operates on vectors by making a diagonal matrix out of its components.

Damping. Each outer iteration solves **??** for a trial step. If the trial step reduces the cost it is accepted and $\lambda \leftarrow 0.1\lambda$; otherwise the step is rejected and retried with $\lambda \leftarrow 10\lambda$. The solver gives up and reports failure if λ exceeds a set limit λ_{max}. Our scheme uses a default value of $\lambda_{max} = 10^7$.

Stopping. Iteration stops when the gradient of the cost is sufficiently small, or when a maximum number of iterations is reached.

0.5 Neural Network Solver

0.5.1 Neural Network Architecture and Training

0.5.2 Choice of loss function

In Section 0.3 we defined two errors: e_p, the positional error, and $e_{\hat{n}}$, the angular error. Now we must define a loss funciton. The only problem is that these errors operate on completely different scales. Given the bounds of our box, e_p varies between 0 and $\sqrt{0.25^2 + 0.25^2 + 0.25^2} \approx 0.433$ and $e_{\hat{n}}$ varies

between 0 and π. To give equal weight to both components we define our loss, $\mathcal{L}$, as the weighted sum between the two as follows:

$$\mathcal{L} = e_p + \frac{\sqrt{0.1875}}{\pi} e_{\hat{n}} \tag{9}$$

This means that, on average, the loss punishes positional and angular error equally.